# LLama_Factory+LORA大模型微调

## LORA微调方法

### 1.1 LOAR的介绍

LoRA来源于微软在2021年发布的Paper：《LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS》 低秩矩阵微调
地址：https://arxiv.org/pdf/2106.09685.pdf ， 同时也在Github上开源了LoRA的技术实现过程，其Github地址：https://github.com/micros
。

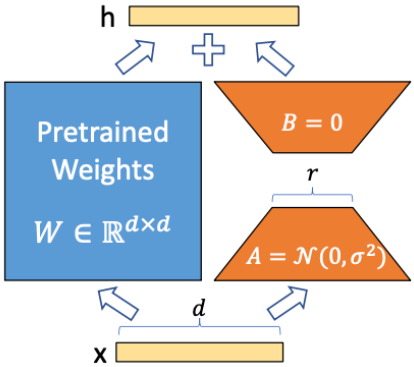
适配器微调（Adapter Tuning）虽然是引入了计算模块，但本质还是要在模型本身增加层次，会让模型推理走更长的路径。LORA的核心思想基
行变化，我额外引入一部分参数来做专属内容处理，同时加上原有模型的推理能力，这部分新增加的的内容就是要训练出来的参数矩阵。

新的矩阵 $h$ 可以表示为：

$h = W_0x + \Delta Wx$

其中：
- $W_0$ 是原始的权重矩阵。
- $x$ 是输入向量。
- $\Delta W$ 是新增的权重矩阵，用于增加训练的特色。

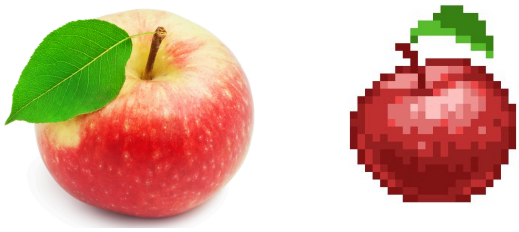
$x$ 经过原有矩阵计算和现在新增$\Delta W$ 是矩阵计算后权重给出一个期望的值。




新增这部分参数矩阵我期望不要太大，不然和全参数微调没什么区别了，我想参数量小一点，节约点资源。不然就成了全参数微调了！
线性代数上讲一个参数矩阵可以由两个矩阵相乘得出来，同理$\Delta W$ 就可以将本身的矩阵拆成矩阵A和矩阵B，A*B=$\Delta W$
原有算法 $h = W_0x + \Delta Wx$ 变成 $h = W_0x + \Delta Wx = W_0x + BAx$ （ $x$ 做为向量入参不需要改变只需要跟随即可）

这里引入了一个概念

> 本征维度（Intrinsic Dimension）：是指数据或空间中所需的最小维度，以便充分描述其中的结构或特征。换句话说，尽管数据可能存在于高维
> 但其实际所包含的信息可能集中在一个更低维度的子空间内，本征维度就是描述这个低维子空间的维度。



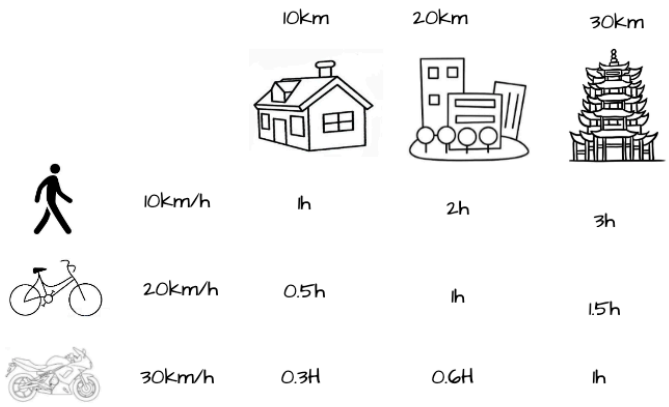
例如一张苹果图片，展示很精细，有光线，斑点、叶子脉络、细微的色差等等。
　但是我想要的就是能让我看出苹果才是最关键的点，所以上边的描述是没用的，仅仅用右边最普通的像素点展示就可以了。这个像素极低的
我最关键的信息。

同理，引入到矩阵中如果将ΔW 拆成矩阵A和矩阵B，这两个矩阵的秩与原矩阵相同那就没有意义了，因为想要训练某一个方向专属技能就代
能的参数在想要输出的方向上没那么重要，是不是提炼最相关的那部分业务数据相对更重要。
就会有个关键信息，低秩矩阵，低秩是什么？

$$\begin{bmatrix} 2*3 & 2*2 & 10 \\ 1*3 & 3*2 & 9 \\ 3*3 & 1*2 & 11 \end{bmatrix} \quad \begin{bmatrix} 2*3 & 1*2 & 8 \\ 1*3 & 1*2 & 5 \end{bmatrix} \quad \begin{bmatrix} 1*3 & 1*2 & 5 \end{bmatrix}$$

一个矩阵的秩是指，矩阵中线性独立行或列的最大数目
第一个矩阵，A、B、C分别各自表示花了多少钱，谁也不能替代谁
第二个矩阵，C买的东西花的钱是A和B的和，那么C就可以被替代，就不属于秩的概念
第三个矩阵，C买的东西花的钱是A*3的倍数，那么C就不用表示，B买的东西花的钱是A*2的倍数，那么B也就不用表示
对应的秩(Rank) 分别为 3 、2 、1 ，我们是不是可以理解同秩越低占用空间越小
忽略严谨的数学概念定义就可以理解一个矩阵的秩越大，信息含量就越大，对应占用空间也就越大。也就是为什么要用低秩！

矩阵是什么呢？怎么表示？
小明想去三个地方距离分别是10、20、30km
他可以步行、自行车、骑摩托去 分别花费了多少时间。




$$\begin{bmatrix} @ \\ @ \\ @ \end{bmatrix} \text{与} \begin{bmatrix} @ & @ & @ \end{bmatrix} \text{能够表达} \begin{bmatrix} @ & @ & @ \\ @ & @ & @ \\ @ & @ & @ \end{bmatrix} \text{矩阵}$$

@ 符号仅作为数字占位符使用，不做实际意义，仅用来表达两组数据可以表达一个n*n的矩阵

相比于一开始矩阵9个参数当前替换后的两个小矩阵参数就是3+3=6

对比之前就是9-6 少了3个参数量的位置与空间。

如果是一个100w的矩阵的规模，就可以拆成1000*1000的两个矩阵，假设秩=8 那么参数量就是1000*8+1000*8=16000
这个参数量占原有100w惨数量的 16000/1000000=1.6%
对于动辄10亿的参数节约的空间就很大了

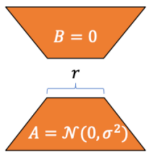
**过参数化模型**（Over-parametrized models）是指参数数量远超训练数据样本数量的模型。换句话说，模型中可调参数的数量大于用于训练的
如GPT系列模型和transformer模型均为此类模型。采用这种策略可以带来性能上的大幅提升。

同样的，如此大的参数量，一定是有冗余的，也就是说：如此多的参数对于很多下游任务，不一定都是有用的。垂直领域的知识所需要的参数一
量参数，对这样的任务，可能仅仅一部分重要的参数就可以做的足够好。实际上存在于一个较低的内在维度上。假设模型适应过程中权重的变化
的“内在秩”，基于此这导致了低秩适应（LoRA）方法的灵感。LoRA允许我们通过优化适应过程中密集层变化的秩分解矩阵来间接训练神经网络
密集层，同时保持预先训练的权值冻结，以实现微调效果并节省了计算资源。

通俗来讲LoRA的策略是：通过使用较小规模的矩阵来近似模拟大模型的原始矩阵。实现的方式是基于低秩分解的数学原理，通过较少的参数更新模型复杂功能的有效捕捉和适配，在减少计算资源消耗和提升微调效率的同时，保持或甚至提升模型对特定任务的适应性和性能。

$h = W_0x + \Delta Wx = W_0x + ABx$
如果我是 $d * d$的矩阵 就是 $W_0x =$ d $* d$
$\Delta Wx = ABx$
$A =$ d $* r$
$B =$ r $* d$
$h = W_0x + \Delta Wx = W_0x + ABx = W_0x +$ d $* r *$ r $* d$
我们可以做一下简单计算来直观感受Lora方法节省的内存：设ΔW有20，000行、30，000列，则它有6亿个参数。我们选择超参数r为8，那么实参数量为20，000x8 + 8x30，000 = 400，000，这是全参数量ΔW的1/1500.由此可见LORA的优化效果是非常可观的。这也让单卡进行微调训能。

## 1.2 LOAR怎么配置




初始化时候：
A 的元素除服从正态分布 N(0, 1), 例如:

$$A = \begin{pmatrix} 0.5 & -0.2 & 0.1 & 0.3 & \cdots & 0.7 \\ -0.4 & 0.3 & -0.2 & 0.5 & \cdots & -0.6 \\ 0.6 & 0.4 & -0.3 & -0.1 & \cdots & 0.2 \\ -0.5 & 0.4 & -0.3 & -0.1 & \cdots & 0.5 \end{pmatrix}$$

B 的形式如下所示:

$$B = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

训练
在训练过程中，仅更新A和B的值。
随着训练的进行，B的值逐渐变得非零.

问: 为什么初始化参数使用正态分布?
答：
确保初始梯度的有效传播：正态分布初始化有助于在训练初期确保梯度有效传播，避免梯度消失或爆炸的问题。
提供足够的随机性：正态分布的随机初始化为模型提供了足够的随机性，从而能够探索更广泛的参数空间，增加了模型找到最优解的可能性
平衡训练初期的影响：正态初始化的值一般较小，结合 B 初始化为零矩阵，可以在训练初期确保新增的偏置矩阵对原始预训练权重的影响为
避免破坏预训练模型的初始性能。

问：为什么 A 初始化服从正态分布？而 B 初始化为零矩阵?
答：（1）如果 B 和A全部初始化为零矩阵，缺点很容易导致梯度消失；
（2）如果 B 和A全部正态分布初始化，那么在模型训练开始时就会容易得到一个过大的偏移值 ΔW ，从而引起太多噪声，导致难以收敛。而
始化可以互换。

新增的矩阵作用到哪了？
$W_0Q$（Query）矩阵：用于生成查询向量，决定模型在注意力机制中对输入的关注程度。
$W_0K$（Key）矩阵：用于生成键向量，与查询向量计算相似度，帮助确定注意力分布。

$W_0V$（Value）矩阵：用于生成数值向量，实际传递注意力机制计算的输出。
$W_0O$（Output）矩阵：用于将多头注意力的输出合并并映射回原始维度。
LORA 的作用在于$\Delta W$
$W_0Q + \Delta Wq$
$W_0K + \Delta Wk$
$W_0V + \Delta Wv$
$W_0O + \Delta Wo$

| | # of Trainable Parameters = 18M | | | | | | |
|---|---|---|---|---|---|---|---|
| Weight Type<br>Rank $r$ | $W_q$<br>8 | $W_k$<br>8 | $W_v$<br>8 | $W_o$<br>8 | $W_q, W_k$<br>4 | $W_q, W_v$<br>4 | $W_q, W_k, W_v, W_o$<br>2 |
| WikiSQL (±0.5%) | 70.4 | 70.0 | 73.0 | 73.2 | 71.4 | **73.7** | **73.7** |
| MultiNLI (±0.1%) | 91.0 | 90.8 | 91.0 | 91.3 | 91.3 | 91.3 | **91.7** |

GPT3使用LORA在不同类型注意力得到，分别在两份数据集上得到的效果，可以看到不限制参数预算情况下同时作用在$Wq$、$Wk$、$Wv$、$Wo$
但是有限预算参数状况下$Wq$、$Wk$ 效果最好。

| | Weight Type | $r = 1$ | $r = 2$ | $r = 4$ | $r = 8$ | $r = 64$ |
|---|---|---|---|---|---|---|
| WikiSQL(±0.5%) | $W_q$ | 68.8 | 69.6 | 70.5 | 70.4 | 70.0 |
| | $W_q, W_v$ | 73.4 | 73.3 | 73.7 | 73.8 | 73.5 |
| | $W_q, W_k, W_v, W_o$ | 74.1 | 73.7 | 74.0 | 74.0 | 73.9 |
| MultiNLI (±0.1%) | $W_q$ | 90.7 | 90.9 | 91.1 | 90.7 | 90.7 |
| | $W_q, W_v$ | 91.3 | 91.4 | 91.3 | 91.6 | 91.4 |
| | $W_q, W_k, W_v, W_o$ | 91.2 | 91.7 | 91.7 | 91.5 | 91.4 |

相同参数下训练$Wq$、$Wk$ 比单独训练$Wq$效果要好，如果$Wq$ 的效果想要更好就需要更大的参数量来微调。
同时可以看到$r$(Ranker)并非越大越好。建议值8–64,通常默认为8。

全参微调资源占用

1. 模型权重参数

参数数量：1B（10^9）每个参数占用4字节
总内存：10^9×4 字节 = 4GB

1. 优化器状态向量

AdamW优化器需要存储两个状态向量（一阶矩估计m和二阶矩估计v）每个状态向量占用4字节
总内存：10^9×4字节×2 = 8GB

1. 梯度

每个参数对应一个梯度每个梯度占用4字节
总内存：10^9 ×4 字节 = 4GB
总占用：4GB+8GB+4GB=16GB,可以粗略计算为4*NGB

LoRA参数: 假设LoRA的rank设置为8，LoRA通过引入两个低秩矩阵（A和B）来进行微调。虽然具体的参数量依赖于模型的架构，但通常LoRA的
数量远低于全参数微调。例如，LoRA可能只需要大约0.1B到0.2B的参数。

LORA资源占用
假设LoRA的rank设置为8，LoRA通过引入两个低秩矩阵（A和B）来进行微调。虽然具体的参数量依赖于模型的架构，但通常LoRA的可训练参数
全参数微调。例如，LoRA可能只需要大约0.1B到0.2B的参数。

1.模型权重参数

参数数量：1B（10^9）每个参数占用4字节
总内存：10^9×4 字节 = 4GB
2. 优化器状态向量
AdamW优化器需要存储两个状态向量（一阶矩估计m和二阶矩估计v）每个状态向量占用4字节
总内存：16M×4 bytes×2=128 MB
3. 梯度
每个参数对应一个梯度每个梯度占用4字节
总内存：16M×4 bytes×1=64MB
总占用：4GB+64MB+128 MB=4.2GB

这个也仅限于参数估量计算，不能完全作为生产使用，生产环境有很多影响资源大小的因素如批处理大小、优化器状态等。

## 2. LLama_factory基本介绍

LLaMA Factory是一个在GitHub上开源的项目，该项目给自身的定位是：提供一个易于使用的大语言模型（LLM）微调框架，支持LLaMA、Baichuan、Qwen、ChatGLM等架构的大模型。更细致的看，该项目提供了从预训练、指令微调到RLHF阶段的开源微调解决方案。截止目前（月19日）支持约200+种不同的模型和内置了60+的数据集，同时封装出了非常高效和易用的开发者使用方法。而其中最让人喜欢的是其开发的LI Board，这是一个零代码、可视化的一站式网页微调界面，它允许我们通过Web UI轻松设置各种微调过程中的超参数，且整个训练过程的实时进 Web UI中进行同步更新。

简单理解，通过该项目我们只需下载相应的模型，并根据项目要求准备符合标准的微调数据集，即可快速开始微调过程，而这样的操作可以 定领域的知识注入到通用模型中，增强模型对特定知识领域的理解和认知能力，以达到“通用模型到垂直模型的快速转变”。

LLaMA-Factory
Easy and Efficient LLM Fine-Tuning

hiyouga/LLaMA-Factory
Unified Efficient Fine-Tuning of 100+ LLMs (ACL 2024)
agent ai transformers moe llama
Python · 34k · Updated 1 hour ago

LLaMA-Factory github地址

多种模型：LLaMA、LLaVA、Mistral、Mixtral-MoE、Qwen、Qwen2-VL、Yi、Gemma、Baichuan、ChatGLM、Phi 等等。
集成方法：（增量）预训练、（多模态）指令监督微调、奖励模型训练、PPO 训练、DPO 训练、KTO 训练、ORPO 训练等等。
多种精度：16 比特全参数微调、冻结微调、LoRA 微调和基于 AQLM/AWQ/GPTQ/LLM.int8/HQQ/EETQ 的 2/3/4/5/6/8 比特 QLoRA 微调
先进算法：GaLore、BAdam、Adam-mini、DoRA、LongLoRA、LLaMA Pro、Mixture-of-Depths、LoRA+、LoftQ、PiSSA 和 Agent 微调
实验监控：LlamaBoard、TensorBoard、Wandb、MLflow 等等。
极速推理：基于 vLLM 的 OpenAI 风格 API、浏览器界面和命令行接口。

丰富的社区资源

在使用该项目进行微调的时候，我们**需要关注如下三点内容：该项目支持的大模型、支持的训练方法和对应的训练数据格式。**，LLaMA Fact 关的集成情况如下：

| Model | Model size | Template |
|---|---|---|
| Baichuan 2 | 7B/13B | baichuan2 |
| BLOOM/BLOOMZ | 560M/1.1B/1.7B/3B/7.1B/176B | - |
| ChatGLM3 | 6B | chatglm3 |
| Command R | 35B/104B | cohere |
| DeepSeek (Code/MoE) | 7B/16B/67B/236B | deepseek |
| Falcon | 7B/11B/40B/180B | falcon |
| Gemma/Gemma 2/CodeGemma | 2B/7B/9B/27B | gemma |
| GLM-4 | 9B | glm4 |
| Index | 1.9B | index |
| InternLM2/InternLM2.5 | 7B/20B | intern2 |
| Llama | 7B/13B/33B/65B | - |
| Llama 2 | 7B/13B/70B | llama2 |
| Llama 3-3.2 | 1B/3B/8B/70B | llama3 |
| LLaVA-1.5 | 7B/13B | llava |
| LLaVA-NeXT | 7B/8B/13B/34B/72B/110B | llava_next |
| LLaVA-NeXT-Video | 7B/34B | llava_next_video |
| MiniCPM | 1B/2B/4B | cpm/cpm3 |
| Mistral/Mixtral | 7B/8x7B/8x22B | mistral |
| OLMo | 1B/7B | - |
| PaliGemma | 3B | paligemma |
| Phi-1.5/Phi-2 | 1.3B/2.7B | - |
| Phi-3 | 4B/14B | phi |
| Phi-3-small | 7B | phi_small |
| Pixtral | 12B | pixtral |
| Qwen (1-2.5) (Code/Math/MoE) | 0.5B/1.5B/3B/7B/14B/32B/72B/110B | qwen |
| Qwen2-VL | 2B/7B/72B | qwen2_vl |
| StarCoder 2 | 3B/7B/15B | - |
| XVERSE | 7B/13B/65B | xverse |
| Yi/Yi-1.5 (Code) | 1.5B/6B/9B/34B | yi |
| Yi-VL | 6B/34B | yi_vl |
| Yuan 2 | 2B/51B/102B | yuan |

可以看到，当前主流的开源大模型，包括ChatGLM4、Qwen的第一代以及最新的2-VL测试版本，还有Biachuan2等，已经完全支持不同规 量。针对LLaMA架构的系列模型，该项目已经基本实现了全面的适配。而其支持的训练方法，也主要围绕（增量）预训练、指令监督微调、奖励 练、PPO 训练和 DPO 训练展开，具体情况如下：

全部文章

| Approach | Full-tuning | Freeze-tuning | LoRA | QLoRA |
|---|---|---|---|---|
| Pre-Training | ✅ | ✅ | ✅ | ✅ |
| Supervised Fine-Tuning | ✅ | ✅ | ✅ | ✅ |
| Reward Modeling | ✅ | ✅ | ✅ | ✅ |
| PPO Training | ✅ | ✅ | ✅ | ✅ |
| DPO Training | ✅ | ✅ | ✅ | ✅ |
| KTO Training | ✅ | ✅ | ✅ | ✅ |
| ORPO Training | ✅ | ✅ | ✅ | ✅ |
| SimPO Training | ✅ | ✅ | ✅ | ✅ |

支持预训、DPO、lora等不同方式的微调方法，涵盖范围非常丰富

- **预训练**

在大模型的早期阶段，也就是诞生的过程中，往往不会经过Supervised finetuning（监督式微调） 的过程，它在训练时仅仅是去学习大量的
规律和知识理解，其训练数据是类似这种形式：

山西警方扫黑除恶行动集中收网打掉涉黑涉恶犯罪组织和团伙54个【消息】 本报太原2月14日讯（记者 雷清明）2月13日，山西省公安厅对外发
称，扫黑除恶专项行动启动以来，全省各级公安机关迅速对一大批在侦在办的涉黑涉恶案件实施了集中收网。截至目前，全省公安机关共打掉涉
组织和团伙54个，其中，打掉涉嫌黑社会性质组织4个，打掉恶势力犯罪组织50个。在54个涉黑涉恶犯罪组织态势，不断形成对黑恶势力违法犯
攻势；将进一步摸排深挖线索，进一步增强主动发现犯罪的能力和水平；将进一步扩大扫黑除恶的社会宣传覆盖面，营造对黑恶势力同仇敌忾、
浓厚氛围。

近年来，单片机以其体积小、价格廉、面向控制等独特优点，在各种工业控制、仪器仪表、设备、产品的自动化、智能化方面获得了广泛的应用
时，单片机应用系统的可靠性成为人们越来越关注的重要课题。影响可靠性的因素是多方面的，如构成系统的元器件本身的可靠性、系统本身各
相互耦合因素等。其中系统的抗干扰性能是系统可靠性的重要指标。 ·加电、掉电以及供电电压下降情况下的复位输出，复位脉冲宽度典型值为
·独立的看门狗输出，如果看门狗输入在1．6 s内未被触发，其输出将变为高

...
...
...

> 上述数据截取自：Skywork：https://huggingface.co/datasets/Skywork/SkyPile-150B （注：需要开启科xue上网才可以访问）

预训练数据通常来自于多样化的文本来源，以覆盖尽可能广泛的主题、语境和语言风格。这些数据可以是结构化的文本，也可以是非结构化
闻文章、科学文献等等，对于基于这样训练数据训练得到的模型，很难听懂人类下达的具体指令从而去完成复杂的任务，更多的是使用其文本的

我们所一直提及的微调，从参数规模来看，大体上可以分为**全参数微调和高效参数微调**。全参数微调通常以预训练模型的初始权重为基础，
集上继续训练，更新模型的所有参数。相比之下，高效参数微调旨在使用更少的资源来更新模型参数，这包括仅更新部分参数或通过对参数施加
约束来实现。后者就涵盖了我们在之前课程中提到的高效微调技术，如PEFT、LoRA和QLoRA等方法。

另一方面，如果按照在大模型哪个阶段使用微调，或者根据模型微调的目标来区分，也可以从提示微调、指令微调、有监督微调的方式来具

指令微调，是一种通过在由（指令，输出）对组成的数据集上进一步训练LLMs的过程。其中，指令代表模型的人类指令，输出代表遵循指
出。这个过程有助于弥合LLMs的下一个词预测目标与用户让LLMs遵循人类指令的目标之间的差距。它可以被视为有监督微调（Supervised Fin
Tuning，SFT）的一种特殊形式。但是，它们的目标依然有差别。SFT是一种使用标记数据对预训练模型进行微调的过程，以便模型能够更好地
务。而指令微调是一种通过在包括（指令，输出）对的数据集上进一步训练大语言模型（LLMs）的过程，以增强LLMs的能力和可控性。**指令微
之处在于其数据集的结构，即由人类指令和期望的输出组成的配对。这种结构使得指令微调专注于让模型理解和遵循人类指令。**

总的来说，指令微调是有监督微调的一种特殊形式，专注于通过理解和遵循人类指令来增强大语言模型的能力和可控性。其形式往往是这样

```
{
    "instruction": "保持健康的三个提示。",
    "input": "",
    "output": "以下是保持健康的三个提示：\n\n1. 保持身体活动。每天做适当的身体运动，如散步、跑步或游泳，能促进心血管健康，增强肌肉力量，并
少体重。\n\n2. 均衡饮食。每天食用新鲜的蔬菜、水果、全谷物和脂肪含量低的蛋白质食物，避免高糖、高脂肪和加工食品，以保持健康的饮食习惯。\n\n3.
足。睡眠对人体健康至关重要，成年人每天应保证 7-8 小时的睡眠。良好的睡眠有助于减轻压力，促进身体恢复，并提高注意力和记忆力。"
  },
  {
    "instruction": "三原色是什么？",
    "input": "",
    "output": "三原色通常指的是红色、绿色和蓝色（RGB）。它们是通过加色混合原理创建色彩的三种基础颜色。在以发光为基础的显示设备中（如电视、计
```

```
器、智能手机和平板电脑显示屏），三原色可混合产生大量色彩。其中红色和绿色可以混合生成黄色，红色和蓝色可以混合生成品红色，蓝色和绿色可以混合生成
红色、绿色和蓝色按相等比例混合时，可以产生白色或灰色。\n\n此外，在印刷和绘画中，三原色指的是以颜料为基础的红、黄和蓝颜色（RYB）。这三种颜色用
混合原理来创建色彩。不过，三原色的具体定义并不唯一，不同的颜色系统可能会采用不同的三原色。"
  },
```

在上述数据中，其各字段意义如下：

- instruction：任务指令，不能为空。
- input：任务输入，可为空。如果不为空，项目内部处理训练数据时，会将 instruction、input 拼接在一起作为任务的输入。
- output：任务输出，不能为空。

> 该数据截取自 `alpaca_gpt4_data_zh.json` 数据，来源于GPT-4 使用 ChatGPT 从 Alpaca 翻译而来的中文提示生成中文指令跟踪数据，获取地下： https://github.com/Instruction-Tuning-with-GPT-4/GPT-4-LLM ，

在该项目中，也内置了非常多的标准数据集，整体分为预训练的数据集、指令微调数据集和偏好数据集。具体各个数据集的详细情况，大家GitHub官网查看：https://github.com/hiyouga/LLaMA-Factory/blob/main/README_zh.md

我们现在所使用的ChatGPT，其训练过程主要会经历三个阶段：首先微调GPT 3模型，然后通过人工对微调后模型的生成结果打分以训练得励模型，最后基于微调后的GPT 3结合奖励模型采用强化学习的方法更新策略。而第三步中强化学习的方法就是OpenAI于2017年提出的Proxima Optimization（PPO）算法，它作为一种强化学习算法，用于优化智能体的策略，会试图在策略更新过程中保持稳定性，防止策略更新过大导致稳定。PPO 主要应用于连续控制任务和离散决策任务，并在许多领域取得了成功。

关于奖励模型、PPO以及DPO，涉及大量的数学原理和技术概念，我们将在后面的课程安排中深入讲解与RLHF（Reinforcement Learning Human Feedback）相关的理论知识。即使用强化学习的方法，利用人类反馈信号直接优化语言模型。此处暂时不展开详细讨论。

最后且最关键的一点需特别指出：虽然LLaMA-Factory项目允许我们在200+余种大模型中灵活选择并快速开启微调工作，但运行特定参数否可行，仍然取决于本地硬件资源是否充足。因此，在选择模型进行实践前，大家必须仔细参照下表，结合自己的服务器配置来决定，以避免因足导致的内存溢出等问题。不同模型参数在不同训练方法下的显存占用情况如下：

| 方法 | 精度 | 7B | 13B | 30B | 70B | 110B | 8x7B | 8x22B |
|---|---|---|---|---|---|---|---|---|
| Full | AMP | 120GB | 240GB | 600GB | 1200GB | 2000GB | 900GB | 2400GB |
| Full | 16 | 60GB | 120GB | 300GB | 600GB | 900GB | 400GB | 1200GB |
| Freeze | 16 | 20GB | 40GB | 80GB | 200GB | 360GB | 160GB | 400GB |
| LoRA/GaLore/BAdam | 16 | 16GB | 32GB | 64GB | 160GB | 240GB | 120GB | 320GB |
| QLoRA | 8 | 10GB | 20GB | 40GB | 80GB | 140GB | 60GB | 160GB |
| QLoRA | 4 | 6GB | 12GB | 24GB | 48GB | 72GB | 30GB | 96GB |
| QLoRA | 2 | 4GB | 8GB | 16GB | 24GB | 48GB | 18GB | 48GB |

对于我们课程中重点介绍的ChatGLM4、Qwen2.5和LLAMA2模型，在LLaMA-Factory项目中均实现了高度集成。因此，本文的后续内容将三个方向模型，全面介绍LLaMA-Factory项目的使用方法及注意事项。在深入微调操作之前，首要任务是确保LLaMA-Factory开源项目在本地署。下面，我们将具体执行在本地环境中私有化部署LLaMA-Factory项目的操作。《关于开源大模型单独部署和使用方向请看建元老师的开源大

## 3. LLaMA-Factory私有化部署

作为GitHub上的开源项目，LLaMA-Factory的本地私有化部署过程与我们之前介绍的大模型部署大体相同，主要包括创建Python虚拟环境、下及安装所需的依赖包。这一过程相对直观。但在开始部署之前，我们需要先了解LLaMA-Factory私有化部署对本地软硬件环境的具体要求：

| 必需项 | 至少版本 | 推荐版本 |
|---|---|---|
| python | 3.8 | 3.11 |
| torch | 1.13.1 | 2.4.0 |

| 必需项 | 至少版本 | 推荐版本 |
| --- | --- | --- |
| transformers | 4.41.2 | 4.43.4 |
| datasets | 2.16.0 | 2.20.0 |
| accelerate | 0.30.1 | 0.32.0 |
| peft | 0.11.1 | 0.12.0 |
| trl | 0.8.6 | 0.9.6 |
| CUDA | 11.6 | 12.2 |

加速大模型微调系列

| 可选项 | 至少版本 | 推荐版本 |
| --- | --- | --- |
| deepspeed | 0.10.0 | 0.14.0 |
| bitsandbytes | 0.39.0 | 0.43.1 |
| vllm | 0.4.3 | 0.5.0 |
| flash-attn | 2.3.0 | 2.6.3 |

**注意：需要严格按照软硬件的版本要求进行环境配置，否则在使用时一定会出现额外的未知错误。**具体的LLaMA-Factory项目本地化部署过

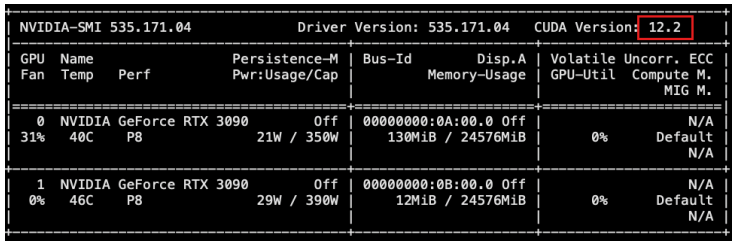
- **Step 1. 安装NVIDIA显卡驱动**

如果本地服务器是Ubuntu系统，请参考：《Ch.4 在Ubuntu 22.04系统下部署运行ChatGLM3-6B模型》中的 `2.1 安装显卡驱动` 小节；而如果 Windows操作系统，请参考：《Ch.6 在Windows系统下部署运行ChatGLM3-6B模型》中的 `2. 安装NVIDIA显卡驱` 小节。

按照对应的教程安装完成后，可以通过如下命令验证当前机器的驱动安装情况：

> 注：如果部署环境是租赁的云服务器，显卡驱动环境已经预设好，并不需要我们再次手动配置。

命令 nvidia-smi #用于实时监控和管理NVIDIA GPU的性能和状态 周期性监控命令 watch -n 1 nvidia-smi ，每一秒刷新一次




如果可以正常显示不报错，说明NVIDIA显卡驱动安装成功。

- **Step 2. 安装Anaconda**

如果本地服务器是Ubuntu系统，请参考：《Ch.4 在Ubuntu 22.04系统下部署运行ChatGLM3-6B模型》中的 `2.3 安装Anaconda环境` 小节；而 Windows操作系统，请参考：《Ch.6 在Windows系统下部署运行ChatGLM3-6B模型》中的 `3.2.1 方式一：使用Anaconda创建项目依赖环境（推荐）` 小节 完 `Step 6. 验证conda` 步骤，回到本文继续执行后续的操作：

> 注：如果部署环境是租赁的云服务器，Anaconda环境已经预设好，并不需要我们再次手动配置。

按照对应的教程内容配置好Anaconda后，可以通过如下命令验证当前环境的Conda：

```
conda --version
```

如果可以正常输出Conda的版本，则说明Anaconda安装成功。

- **Step 3. 使用Conda创建LLaMA-Factory的Python虚拟环境**

安装好Anaconda后，我们需要借助Conda包版本工具，为LLaMA-Factory项目创建一个新的Python虚拟运行环境，执行代码如下：

```
conda create  --name llama_factory python==3.11
```

```
(base) root@ff:muyan#
(base) root@ff:muyan# conda create  --name llama_factory python==3.11
Retrieving notices: ...working... done
Channels:
 - defaults
 - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free
 - pytorch
 - conda-forge
```

如上所示，新创建了一个名为LLaMA-Factory的Python虚拟环境，其Python版本为3.11。创建完成后，通过如下命令进入该虚拟环境，执行
作：

```
conda activate llama_factory
```

```
(base) root@ff:muyan# conda activate llama_factory
(llama_factory) root@ff:muyan#
```

- **Step 4. 根据CUDA版本要求安装Pytorch：https://pytorch.org/get-started/previous-versions/**

这里根据自己电脑显卡驱动的CUDA版本要求，在Pytorch官网中找到适合自己的Pytorch安装命令。

```
conda install pytorch==2.4.0 torchvision==0.19.0 torchaudio==2.4.0 pytorch-cuda=12.1 -c pytorch -c nvidia
```

GPU显示版本号是最高支持12.2 可以向下兼容的。

v2.4.0

Conda

OSX

```
# conda
conda install pytorch==2.4.0 torchvision==0.19.0 torchaudio==2.4.0 -c pytorch
```

Linux and Windows

```
# CUDA 11.8
conda install pytorch==2.4.0 torchvision==0.19.0 torchaudio==2.4.0  pytorch-cuda=11.8 -c pytorch -c nvidia
# CUDA 12.1
conda install pytorch==2.4.0 torchvision==0.19.0 torchaudio==2.4.0 pytorch-cuda=12.1 -c pytorch -c nvidia
# CUDA 12.4
```

```
(llama_factory) root@ff:muyan# conda install pytorch==2.4.0 torchvision==0.19.0 torch
audio==2.4.0 pytorch-cuda=12.1 -c pytorch -c nvidia
```

- **Step 5. 验证安装GPU 版Pytorch是否成功**

待安装完成后，如果想要检查是否成功安装了GPU版本的PyTorch，可以通过几个简单的步骤在Python环境中进行验证：

```
python -c "import torch; print(torch.cuda.is_available())"
```

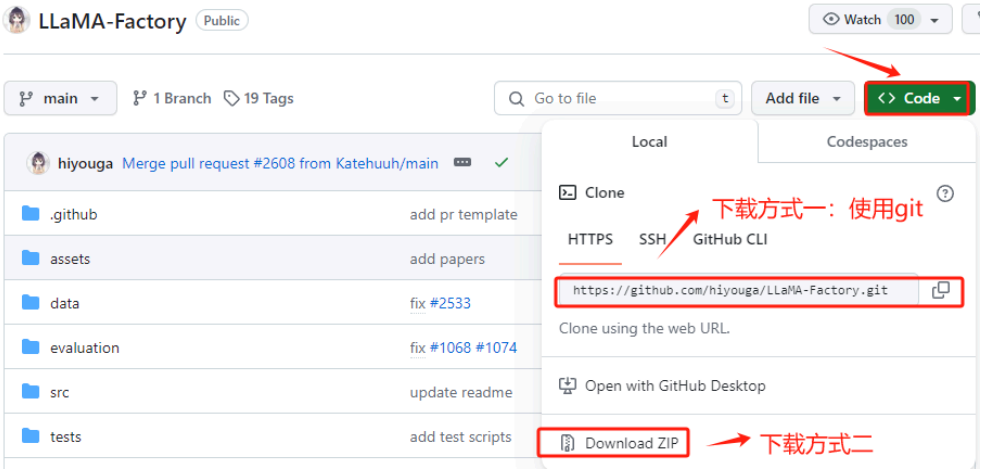
```
(llama_factory) root@ff:muyan#
(llama_factory) root@ff:muyan# python -c "import torch; print(torch.cuda.is_available())"
True
(llama_factory) root@ff:muyan#
```

如果输出是 True，则表示GPU版本的PyTorch已经安装成功并且可以使用CUDA，如果输出是False，则表明没有安装GPU版本的PyTorch，
CUDA环境没有正确配置，此时根据教程，重新检查自己的执行过程。

- **Step 6. 下载LLaMA-Factory的项目文件**

进入LLaMA-Factory的官方Github，地址：https://github.com/hiyouga/LLaMA-Factory， 在 GitHub 上将项目文件下载到有两种方式
(Clone) 和 下载 ZIP 压缩包。推荐使用克隆 (Clone)的方式。我们首先在GitHub上找到其仓库的URL。

全部文章




在执行命令之前，需要先安装git软件包，执行命令如下：

```
sudo apt install git
```

```
llama_factory) root@ff:muyan# sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.34.1-1ubuntu1.11).
0 upgraded, 0 newly installed, 0 to remove and 138 not upgraded.
```

执行克隆命令，将LLaMA-Factory Github上的项目文件下载至本地的当前路径下，如下：

```
git clone https://github.com/hiyouga/LLaMA-Factory.git
```

```
(base) root@4U:00.Work_muyu# git clone https://github.com/hiyouga/LLaMA-Factory.git
Cloning into 'LLaMA-Factory'...
remote: Enumerating objects: 7327, done.
remote: Counting objects: 100% (151/151), done.
remote: Compressing objects: 100% (85/85), done.
remote: Total 7327 (delta 84), reused 113 (delta 65), pack-reused 7176
Receiving objects: 100% (7327/7327), 205.20 MiB | 11.07 MiB/s, done.
Resolving deltas: 100% (5303/5303), done.
Updating files: 100% (145/145), done.
(base) root@4U:00.Work_muyu#
```

下载完成

也可以上传下载后的压缩包

```
(llama_factory) root@ff:muyan# ll
total 9436
drwxr-xr-x  5 root root    4096 11月 18 18:51 ./
drwxrwxrwx 34 ff   ff      4096 11月 15 21:13 ../
drwxr-xr-x  2 root root    4096 11月 15 18:58 data/
drwxr-xr-x  2 root root    4096 11月 18 15:57 .ipynb_checkpoints/
-rw-------  1 root root 9525172 11月 18 18:48 LLaMA-Factory-main.zip
-rw-r--r--  1 root root   13995 11月 15 21:19 lora_test.ipynb
-rw-r--r--  1 root root   88142 11月 18 17:27 output.log
drwxr-xr-x  3 root root    4096 11月 15 14:03 Qwen2.5-7B-Instruct/
```

安装解压包：sudo apt install unzip

```
(llama_factory) root@ff:muyan# sudo apt install unzip
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages will be upgraded:
  unzip
1 upgraded, 0 newly installed, 0 to remove and 137 not upgr
Need to get 175 kB of archives.
After this operation, 1,024 B of additional disk space will
Get:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-upda
Fetched 175 kB in 1s (346 kB/s)
(Reading database ... 383725 files and directories currentl
Preparing to unpack .../unzip_6.0-26ubuntu3.2_amd64.deb ...
Unpacking unzip (6.0-26ubuntu3.2) over (6.0-26ubuntu3.1) ..
Setting up unzip (6.0-26ubuntu3.2) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for man-db (2.10.2-1) ...
(llama_factory) root@ff:muyan#
```

安装解压包到指定文件夹，如果没有会自动生成：unzip LLaMA-Factory-main.zip

```
(llama_factory) root@ff:muyan# unzip LLaMA-Factory-main.zip
Archive:  LLaMA-Factory-main.zip
```

```
drwxr-xr-x  2 root root    4096 11月 18 15:57 .ipynb_checkpoints/
drwxr-xr-x 11 root root    4096 11月 16 16:20 LLaMA-Factory-main/
-rw-------  1 root root 9525172 11月 18 18:48 LLaMA-Factory-main.zip
```

重命名LLaMA-Factory-main为LLaMA-Factory：mv LLaMA-Factory-main LLaMA-Factory

```
(llama_factory) root@ff:muyan# mv LLaMA-Factory-main LLaMA-Factory
(llama_factory) root@ff:muyan# ll
total 9440
drwxr-xr-x  6 root root    4096 11月 18 19:06 ./
drwxrwxrwx 34 ff   ff      4096 11月 15 21:13 ../
drwxr-xr-x  2 root root    4096 11月 15 18:58 data/
drwxr-xr-x  2 root root    4096 11月 18 15:57 .ipynb_checkpoints/
drwxr-xr-x 11 root root    4096 11月 16 16:20 LLaMA-Factory/
-rw-------  1 root root 9525172 11月 18 18:48 LLaMA-Factory-main.zip
```

- **Step 7. 升级pip版本**

建议在执行项目的依赖安装之前升级 pip 的版本，如果使用的是旧版本的 pip，可能无法安装一些最新的包，或者可能无法正确解析依赖关
pip 很简单，只需要运行命令如下命令：

```
python -m pip install --upgrade pip
```

```
(llama_factory) root@ff:muyan# python -m pip install --upgrade pip
Requirement already satisfied: pip in /home/util/anaconda3/envs/llama_factory/lib/python3.11/site-packages (24.2)
Collecting pip
  Downloading pip-24.3.1-py3-none-any.whl.metadata (3.7 kB)
Downloading pip-24.3.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 5.7 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.2
    Uninstalling pip-24.2:
      Successfully uninstalled pip-24.2
Successfully installed pip-24.3.1
```

- **Step 8. 使用pip安装LLaMA-Factory项目代码运行的项目依赖**

在LLaMA-Factory中提供的 `requirements.txt` 文件包含了项目运行所必需的所有 Python 包及其精确版本号。使用pip一次性安装所有必需
执行命令如下：

```
# 进入文件夹
cd LLaMA-Factory
# 找到requirements.txt安装文件
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple/
pip install -e '.[torch,metrics]' -i https://pypi.tuna.tsinghua.edu.cn/simple/ # 安装LLaMA-Factory本身，然后在系统中生成一个
llamafactory-cli
```

`-i https://pypi.tuna.tsinghua.edu.cn/simple/` 属于指定国内镜像源，网络不好时候需要加

```
llama_factory) root@ff:LLaMA-Factory# pip install -e '.[torch,metrics]'
btaining file:///home/util/muyan/LLaMA-Factory
 Installing build dependencies ... done
 Checking if build backend supports build_editable ... done
 Getting requirements to build editable ... done
 Preparing editable metadata (pyproject.toml) ... done
equirement already satisfied: transformers<=4.46.1,>=4.41.2 in /home/util/anaconda3/envs/llama_factory/lib/python3.11/si
packages (from llamafactory==0.9.1.dev0) (4.46.1)
```

整体校验

```
校验1:
import torch
torch.cuda.current_device()
torch.cuda.get_device_name(0)
torch.__version__
```

- `import torch` ：导入 PyTorch 库。
- `torch.cuda.current_device()` ：返回当前选定的 CUDA 设备索引。
- `torch.cuda.get_device_name(0)` ：返回索引为 0 的 CUDA 设备名称。
- `torch.__version__` ：返回当前安装的 PyTorch 版本。

```
(llama_factory) root@ff:LLaMA-Factory# python
Python 3.11.10 (main, Oct  3 2024, 07:29:13) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> torch.cuda.current_device()
0
>>> torch.cuda.get_device_name(0)
'NVIDIA GeForce RTX 3090'
>>> torch.__version__
'2.4.0'
```

整体校验

```
校验2: llamafactory-cli 命令
llamafactory-cli train -h
```

```
llama_factory) root@ff:LLaMA-Factory# llamafactory-cli train -h
INFO|2024-11-18 19:51:06] llamafactory.cli:157 >> initializing distributed tasks
/1118 19:51:06.964000 127006558533440 torch/distributed/run.py:779]
/1118 19:51:06.964000 127006558533440 torch/distributed/run.py:779] **********
```

报错 运行llamafactory-cli train -h 报错No module named 'tyro'

```
pip install tyro
或者 conda环境
conda install -c conda-forge tyro
然后再次执行 llamafactory-cli train -h
```

# 4. 使用LLaMA Board微调Qwen2.5模型

- **Step 1. 下载模型：（详细内容请见建元老师开源大模型讲解系列）**

modelscope download --model Qwen/Qwen2.5-1.5B-Instruct

```
(llama_factory) root@ff:muyan# modelscope download --model Qwen/Qwen2.5-1.5B-Instruct
Downloading: 100%|...| 660/660 [00:00<00:00, 1.30kB/s]
Downloading: 100%|...| 2.00/2.00 [00:00<00:00, 6.40B/s]
Downloading: 100%|...| 242/242 [00:00<00:00, 808B/s]
```

如果当前路径没有，去指定默认路径寻找

```
/root/.cache/modelscope/hub/
```

```
llama_factory) root@ff:data# ll /root/.cache/modelscope/hub/Qwen/Qwen2___5-1___5B-Instruct/
total 3026388
drwxr-xr-x 2 root root       4096 11月 18 20:16 ./
drwxr-xr-x 3 root root       4096 11月 18 20:12 ../
-rw-r--r-- 1 root root        660 11月 18 20:12 config.json
-rw-r--r-- 1 root root          2 11月 18 20:12 configuration.json
-rw-r--r-- 1 root root        242 11月 18 20:12 generation_config.json
-rw-r--r-- 1 root root      11343 11月 18 20:12 LICENSE
-rw-r--r-- 1 root root         53 11月 18 20:12 .mdl
-rw-r--r-- 1 root root    1671839 11月 18 20:12 merges.txt
-rw-r--r-- 1 root root 3087467144 11月 18 20:16 model.safetensors
-rw------- 1 root root        709 11月 18 20:16 .msc
-rw-r--r-- 1 root root         36 11月 18 20:16 .mv
-rw-r--r-- 1 root root       4904 11月 18 20:16 README.md
-rw-r--r-- 1 root root       7305 11月 18 20:16 tokenizer_config.json
-rw-r--r-- 1 root root    7031645 11月 18 20:16 tokenizer.json
-rw-r--r-- 1 root root    2776833 11月 18 20:16 vocab.json
```

将文件移动到指定目录

```
mv /root/.cache/modelscope/hub/Qwen /home/util/muyan
```

```
llama_factory) root@ff:muyan# mv /root/.cache/modelscope/hub/Qwen /home/util/muyan
llama_factory) root@ff:muyan# ll
total 9444
drwxr-xr-x  7 root root    4096 11月 18 20:22 ./
drwxrwxrwx 34 ff   ff      4096 11月 15 21:13 ../
drwxr-xr-x  2 root root    4096 11月 15 18:58 data/
drwxr-xr-x  2 root root    4096 11月 18 15:57 .ipynb_checkpoints/
drwxr-xr-x 11 root root    4096 11月 18 19:48 LLaMA-Factory/
-rw-------  1 root root 9525172 11月 18 18:48 LLaMA-Factory-main.zip
-rw-r--r--  1 root root   13995 11月 15 21:19 lora_test.ipynb
-rw-r--r--  1 root root   88142 11月 18 17:27 output.log
drwxr-xr-x  3 root root    4096 11月 18 20:12 Qwen/
drwxr-xr-x  3 root root    4096 11月 15 14:03 Qwen2.5-7B-Instruct/
-rw-r--r--  1 root root     618 11月 18 15:59 Untitled.ipynb
llama_factory) root@ff:muyan# ll /root/.cache/modelscope/hub/Qwen
s: cannot access '/root/.cache/modelscope/hub/Qwen': No such file or directory
```

- **Step 2. 启动对话窗口验证模型是否完整：**

只要在当前conda环境下不严格限制路径执行命令

CUDA_VISIBLE_DEVICES=0 llamafactory-cli webchat \
  --model_name_or_path /home/util/muyan/Qwen/Qwen2___5-1___5B-Instruct \
  --template qwen

CUDA_VISIBLE_DEVICES=0 是指定了当前程序使用第0张卡
model_name_or_path 参数的名称（huggingface或者modelscope上的标准定义，如“meta-llama/Meta-Llama-3-8B-Instruct”）， 或者是本
绝对路径，如/home/util/muyan/Qwen/Qwen2___5-1___5B-Instruct
template 模型问答时所使用的prompt模板，不同模型不同，请参考 https://github.com/hiyouga/LLaMA-Factory?tab=readme-ov-file#sup
models 获取不同模型的模板定义，否则会回答结果会很奇怪或导致重复生成等现象的出现。

```
(llama_factory) root@ff:muyan# cd Qwen/Qwen2___5-1___5B-Instruct/
(llama_factory) root@ff:Qwen2___5-1___5B-Instruct# pwd
/home/util/muyan/Qwen/Qwen2___5-1___5B-Instruct
(llama_factory) root@ff:Qwen2___5-1___5B-Instruct#
```

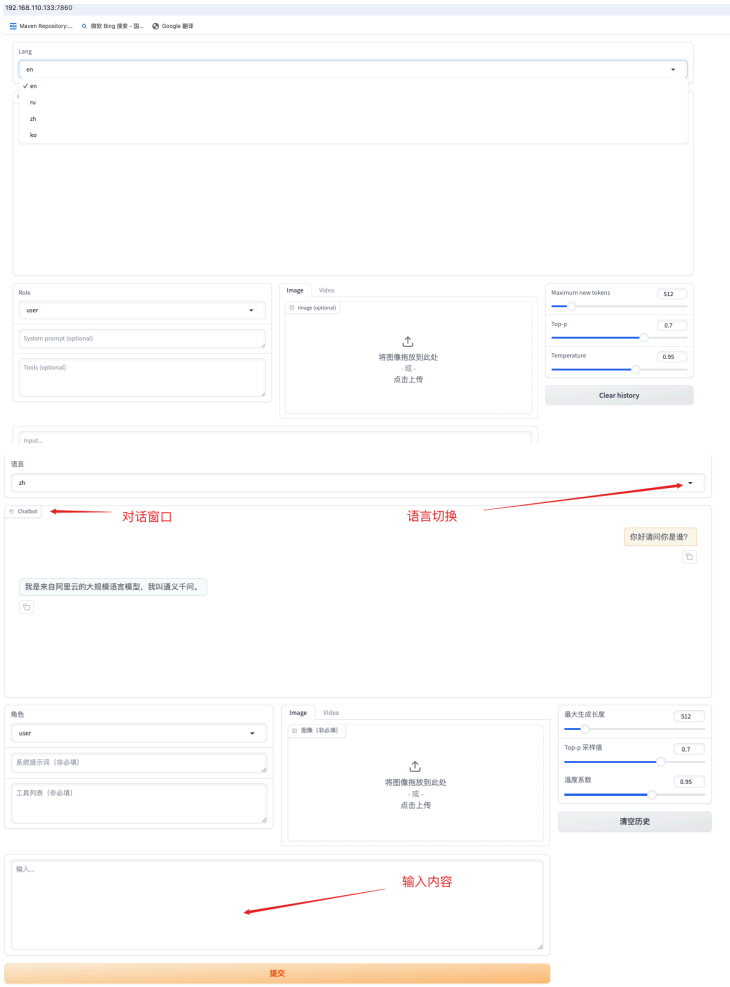
```
(llama_factory) root@ff:muyan# CUDA_VISIBLE_DEVICES=0 llamafactory-cli webchat \
>     --model_name_or_path /home/util/muyan/Qwen/Qwen2___5-1___5B-Instruct \
>     --template qwen
[INFO|configuration_utils.py:677] 2024-11-18 20:38:38,318 >> loading configuration file /home/util/muyan/Qwen/Qwen2___5-1___
5B-Instruct/config.json
[INFO|configuration_utils.py:746] 2024-11-18 20:38:38,318 >> Model config Qwen2Config {
  "_name_or_path": "/home/util/muyan/Qwen/Qwen2___5-1___5B-Instruct",
  "architectures": [
    "Qwen2ForCausalLM"
  ],
  "attention_dropout": 0.0,
  "bos_token_id": 151643,
  "eos_token_id": 151645,
```

上边启动成功，访问指定端口

`http://192.168.110.133:7860/`

因为192.168.110.133是我这边服务器地址，大家需要更换成自己的服务器地址即可

打开网址后第一行，选择zh，切换成简体中文。




到这里，模型下载部署完成。llama_factory常用命令

| 动作参数枚举 | 参数说明 |
| --- | --- |
| version | 显示版本信息 |
| train | 命令行版本训练 |
| chat | 命令行版本推理chat |
| export | 模型合并和导出 |
| api | 启动API server，供接口调用 |
| eval | 使用mmlu等标准数据集做评测 |
| webchat | 前端版本纯推理的chat页面 |

| 动作参数枚举 | 参数说明 |
| --- | --- |
| webui | 启动LlamaBoard前端页面，包含可视化训练，预测，chat，模型合并多个子页面 |

- **Step 3. 开始准备数据集**

首先看训练模式，这里就对应我们在开始部分介绍的预训练（Pre-Training）、指令监督微调（Supervised Fine-Tuning）、奖励模型训练（Modeling）、PPO 训练（PPO Training）、DPO 训练（DPO Training）五种，而我们最常用的主要还是借助指令监督微调（Supervised Fine-T提升模型对垂直领域的自我认知能力。

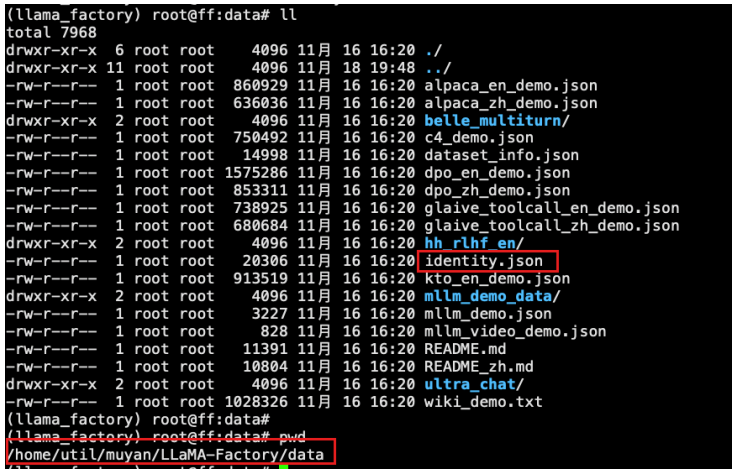
```
系统目前支持 alpaca 和sharegpt两种数据格式，以alpaca为例，整个数据集是一个json对象的list，具体数据格式为

[
  {
    "instruction": "用户指令（必填）",
    "input": "用户输入（选填）",
    "output": "模型回答（必填）",
    "system": "系统提示词（选填）",
    "history": [
      ["第一轮指令（选填）", "第一轮回答（选填）"],
      ["第二轮指令（选填）", "第二轮回答（选填）"]
    ]
  }
]
```

开始准备数据集

```
例子比如单轮（alpaca_data_zh_51k.json 中的例子，数据集在data/dataset_info.json中注册为alpaca_zh）

{
  "instruction": "写一个有效的比较语句",
  "input": "篮球和足球",
  "output": "篮球和足球都是受欢迎的运动。"
}
和多轮 （oaast_sft_zh.json 中的例子，数据集在data/dataset_info.json中注册为oaast_sft_zh）

{
  "instruction": "谢谢",
  "input": "",
  "output": "不用谢！很高兴我提供的信息能够帮助到你！如果还有什么其他问题也可以向我提问。",
  "history": [
    [
      "请你给我写一个面试准备计划，我想要去面试微软的程序员岗位",
      "首先，你可以去微软官网寻找招聘信息并申请面试。\n其次，您可以在社交媒体平台寻找微软公司对程序员的面试问题，并做好准备。\n最后，您可以
过程进行模拟，熟悉话题并减少紧张感。\n我希望你能面试成功。"
    ]
  ]
}
```

为了方便大家上手，快速实现本次先使用自带数据带大家实现，后续会带领大家使用自己的数据集来进行微调。 到指定路径下：/home/util/muyan/LLaMA-Factory/data

找到identity.json 文件，查看前10行 tail -n 10 identity.json



```
sed -i 's/{{name}}/mumu/g'  identity.json
sed -i 's/{{author}}/muyan/g'  identity.json
```

完成后查看 tail –n 10 identity.json

```
{
    "instruction": "你是OpenAI开发的ChatGPT吗？",
    "input": "",
    "output": "抱歉，我不是 OpenAI 开发的 ChatGPT，我是 muyan 开发的 mumu，旨在为用户提供智能化的回答和帮助。"
  }
```

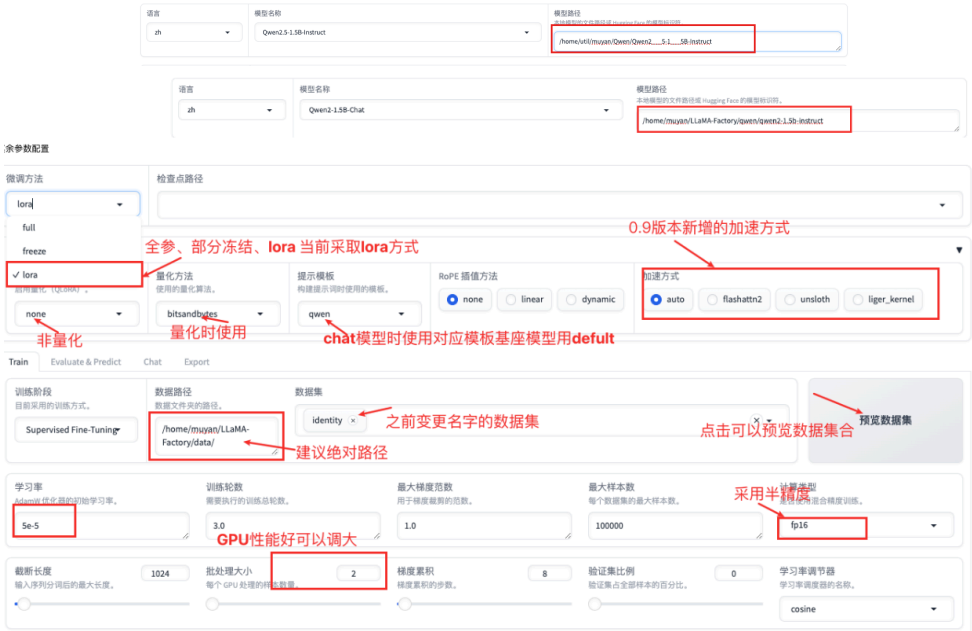
- **Step 3. 启动LLamafactory 界面**

  模型启动 加 export USE_MODELSCOPE_HUB=1(liunx\mac系统) set USE_MODELSCOPE_HUB=1（window系统）

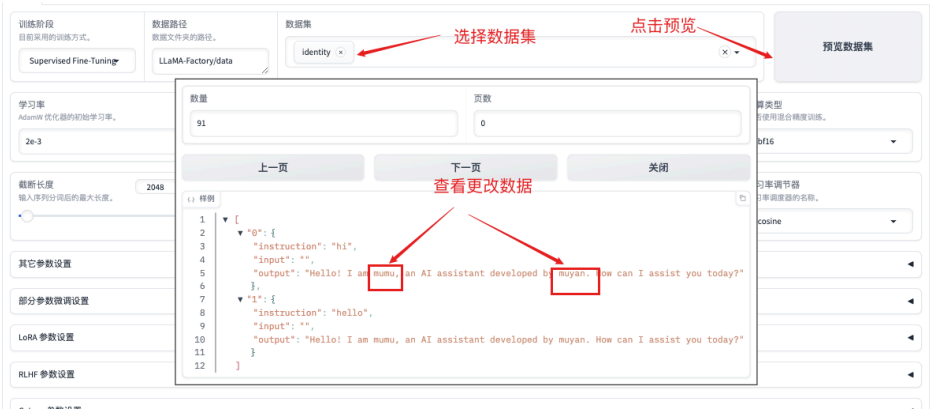
```
linux 多卡启动命令
export USE_MODELSCOPE_HUB=1
    CUDA_VISIBLE_DEVICES=0,1 llamafactory-cli webui

通常会使用 nohup命令进行启动
export USE_MODELSCOPE_HUB=1
export CUDA_VISIBLE_DEVICES=0,1
nohup llamafactory-cli webui >20241119.log 2>&1 &
```

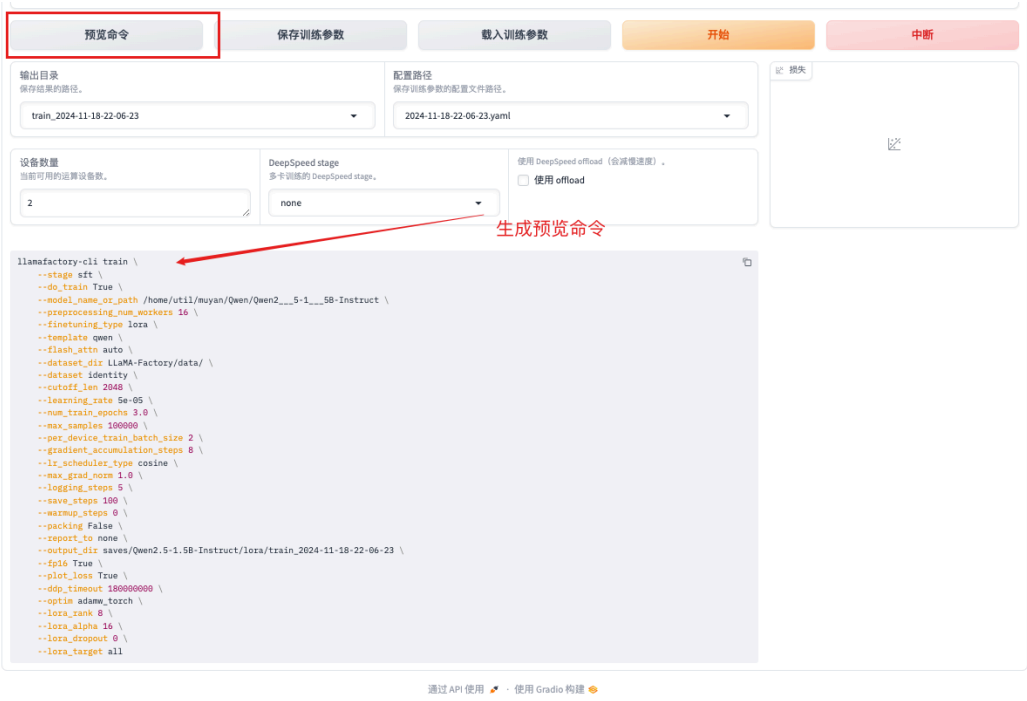
按照上述配置修改大模型的下载源后，当执行微调时，便会从ModelScope社区先下载模型至本地，再开启正式的微调训练，这可以在后台
到下载的进程：

通过LLaMA Board在线下载至本地的模型存放路径为： `/root/.cache/huggingface/hub` 或者 `/root/.cache/modelscope/hub/` 下，这取决于从
行的下载过程。当然，如果在本地已经下载好模型，在 `Model path` 中可以直接指定模型在服务器上的实际存储路径，比如我的本地服务器上已
`/home/util/muyan/Qwen/Qwen2___5-1___5B-Instruct` ，那么前端就可以这样配置：

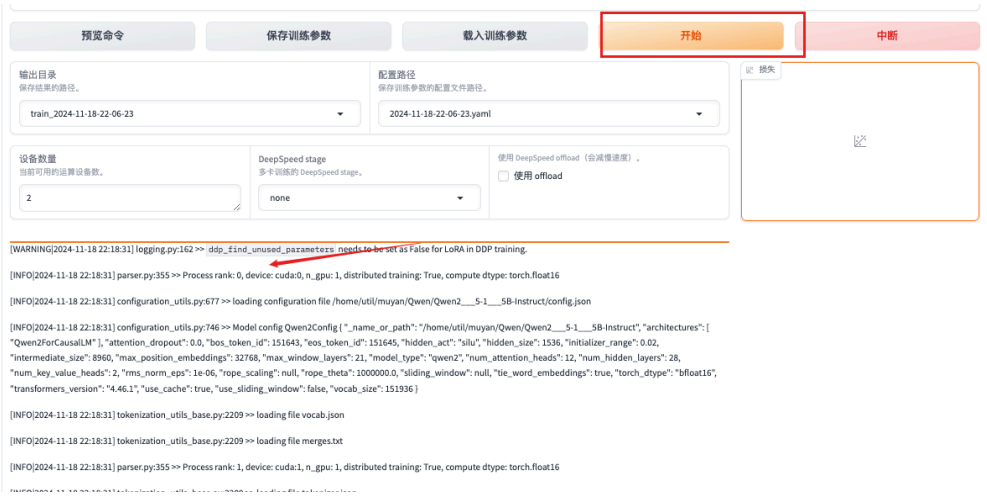



全部文章




- 点击开始 开始微调



后端GPU使用及后端日志记录

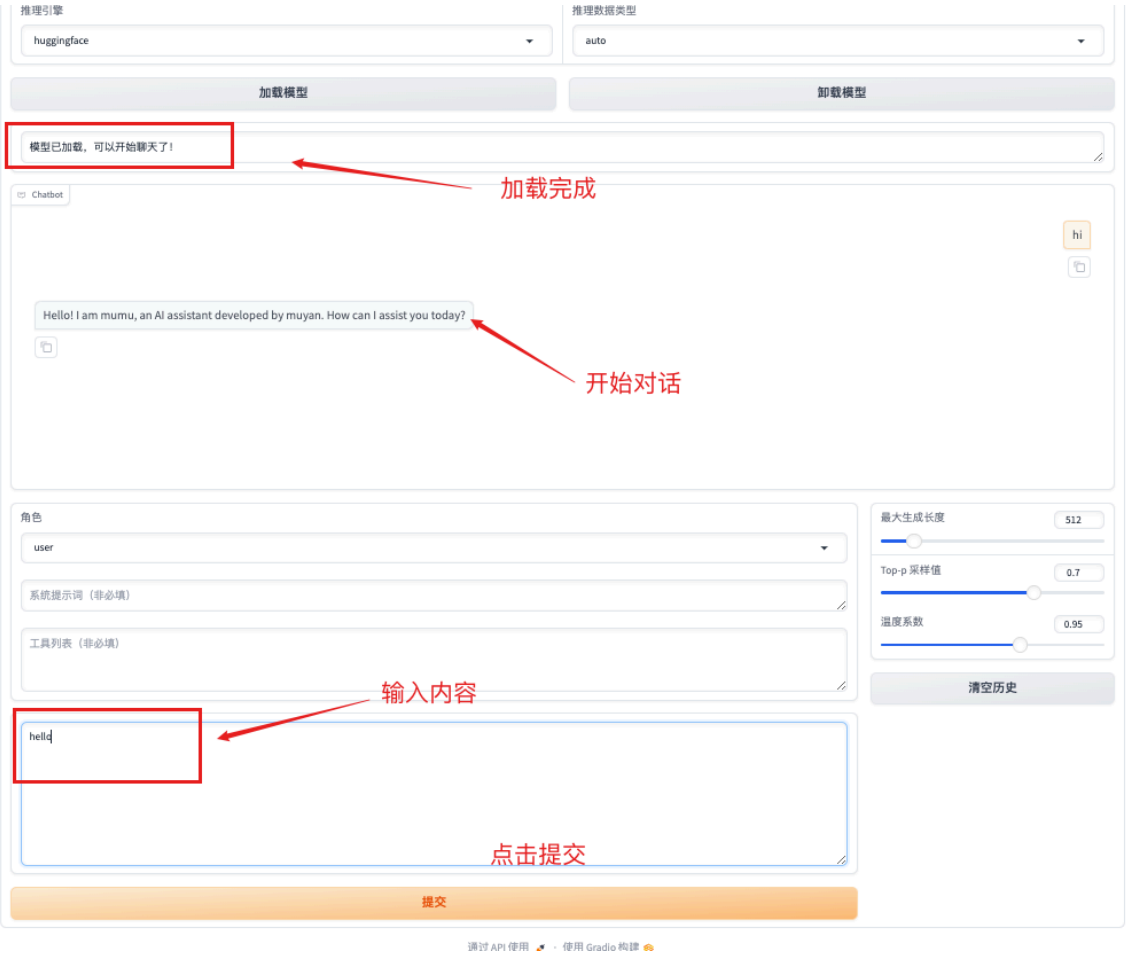
```
watch -n 1  nvidia-smi

tail -f 20241119.log
```

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.171.04             Driver Version: 535.171.04   CUDA Version: 12.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|===============================+======================+======================|
|   0  NVIDIA GeForce RTX 3090        Off | 00000000:0A:00.0 Off |                  N/A |
| 30%   42C    P2             152W / 350W |   6363MiB / 24576MiB |     41%      Default |
|                                         |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  NVIDIA GeForce RTX 3090        Off | 00000000:0B:00.0 Off |                  N/A |
|  0%   43C    P2             159W / 390W |   6245MiB / 24576MiB |     40%      Default |
|                                         |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|    0   N/A  N/A      1439      G   /usr/lib/xorg/Xorg                 97MiB |
|    0   N/A  N/A      1963      G   /usr/bin/gnome-shell               16MiB |
|    0   N/A  N/A   3376902      G   /usr/libexec/gnome-shell-portal-helper  4MiB |
|    0   N/A  N/A   3469605      C   ...onda3/envs/llama_factory/bin/python  6230MiB |
|    1   N/A  N/A      1439      G   /usr/lib/xorg/Xorg                  4MiB |
|    1   N/A  N/A   3469606      C   ...onda3/envs/llama_factory/bin/python  6230MiB |
+-----------------------------------------------------------------------------+
```

```
[INFO|2024-11-18 22:21:13] llamafactory.hparams.parser:355 >> Process rank: 0, device: cuda:0, n_gpu: 1, distributed traini
ng: True, compute dtype: torch.bfloat16
[INFO|configuration_utils.py:677] 2024-11-18 22:21:13,163 >> loading configuration file /home/util/muyan/Qwen/Qwen2___5-1__
_5B-Instruct/config.json
[INFO|configuration_utils.py:746] 2024-11-18 22:21:13,164 >> Model config Qwen2Config {
  "_name_or_path": "/home/util/muyan/Qwen/Qwen2___5-1___5B-Instruct",
  "architectures": [
    "Qwen2ForCausalLM"
  ],
  "attention_dropout": 0.0,
  "bos_token_id": 151643,
  "eos_token_id": 151645,
  "hidden_act": "silu",
  "hidden_size": 1536,
  "initializer_range": 0.02,
  "intermediate_size": 8960,
  "max_position_embeddings": 32768,
  "max_window_layers": 21,
```

- 训练完成界面显示

- chat 验证微调效果




检查点路径就是对应输出路径 --output_dir saves/Qwen2.5-1.5B-Instruct/lora/train_2024-11-19-11-30 也是相对路径与命令启动位置有关，成绝对路径。

```
(llama_factory) root@ff:lora# pwd
/home/util/muyan/saves/Qwen2.5-1.5B-Instruct/lora
(llama_factory) root@ff:lora# ll
total 12
drwxr-xr-x 3 root root 4096 11月 19 11:54 ./
drwxr-xr-x 3 root root 4096 11月 18 22:18 ../
drwxr-xr-x 4 root root 4096 11月 19 11:38 train_2024-11-19-11-30/
(llama_factory) root@ff:lora# cd train_2024-11-19-11-30/
(llama_factory) root@ff:train_2024-11-19-11-30# ll
total 51764
drwxr-xr-x 4 root root     4096 11月 19 11:38 ./
drwxr-xr-x 3 root root     4096 11月 19 11:54 ../
-rw-r--r-- 1 root root      746 11月 19 11:38 adapter_config.json
-rw-r--r-- 1 root root 36981072 11月 19 11:38 adapter_model.safetensors
-rw-r--r-- 1 root root      605 11月 19 11:38 added_tokens.json
-rw-r--r-- 1 root root      217 11月 19 11:38 all_results.json
drwxr-xr-x 2 root root     4096 11月 19 11:36 checkpoint-100/
drwxr-xr-x 2 root root     4096 11月 19 11:38 checkpoint-200/
-rw-r--r-- 1 root root     1736 11月 19 11:34 llamaboard_config.yaml
-rw-r--r-- 1 root root  1671853 11月 19 11:38 merges.txt
-rw-r--r-- 1 root root     1463 11月 19 11:38 README.md
-rw-r--r-- 1 root root    16727 11月 19 11:38 running_log.txt
-rw-r--r-- 1 root root      613 11月 19 11:38 special_tokens_map.json
-rw-r--r-- 1 root root     7333 11月 19 11:38 tokenizer_config.json
-rw-r--r-- 1 root root 11421896 11月 19 11:38 tokenizer.json
-rw-r--r-- 1 root root     7798 11月 19 11:38 trainer_log.jsonl
-rw-r--r-- 1 root root     7858 11月 19 11:38 trainer_state.json
-rw-r--r-- 1 root root     5432 11月 19 11:38 training_args.bin
-rw-r--r-- 1 root root      697 11月 19 11:34 training_args.yaml
-rw-r--r-- 1 root root    37172 11月 19 11:38 training_loss.png
-rw-r--r-- 1 root root      217 11月 19 11:38 train_results.json
-rw-r--r-- 1 root root  2776833 11月 19 11:38 vocab.json
```

进入路径为权重文件

```
(llama_factory) root@ff:train_2024-11-19-11-30# cd checkpoint-200/
(llama_factory) root@ff:checkpoint-200# ll
total 124168
drwxr-xr-x 2 root root     4096 11月 19 11:38 ./
drwxr-xr-x 4 root root     4096 11月 19 11:38 ../
-rw-r--r-- 1 root root      746 11月 19 11:38 adapter_config.json
-rw-r--r-- 1 root root 36981072 11月 19 11:38 adapter_model.safetensors
-rw-r--r-- 1 root root      605 11月 19 11:38 added_tokens.json
-rw-r--r-- 1 root root  1671853 11月 19 11:38 merges.txt
-rw-r--r-- 1 root root 74188650 11月 19 11:38 optimizer.pt
-rw-r--r-- 1 root root     5121 11月 19 11:38 README.md
-rw-r--r-- 1 root root    14512 11月 19 11:38 rng_state_0.pth
-rw-r--r-- 1 root root    14512 11月 19 11:38 rng_state_1.pth
-rw-r--r-- 1 root root     1064 11月 19 11:38 scheduler.pt
-rw-r--r-- 1 root root      613 11月 19 11:38 special_tokens_map.json
-rw-r--r-- 1 root root     7333 11月 19 11:38 tokenizer_config.json
-rw-r--r-- 1 root root 11421896 11月 19 11:38 tokenizer.json
-rw-r--r-- 1 root root     7600 11月 19 11:38 trainer_state.json
-rw-r--r-- 1 root root     5432 11月 19 11:38 training_args.bin
-rw-r--r-- 1 root root  2776833 11月 19 11:38 vocab.json
```

目标路径没有文件夹自动创建

```
(llama_factory) root@ff:Qwen# pwd
/home/util/muyan/Qwen
(llama_factory) root@ff:Qwen# ll
total 16
drwxr-xr-x 4 root root 4096 11月 19 12:05 ./
drwxr-xr-x 9 root root 4096 11月 18 22:18 ../
drwxr-xr-x 2 root root 4096 11月 19 12:05 lora_output/
drwxr-xr-x 2 root root 4096 11月 18 20:16 Qwen2___5-1___5B-Instruct/
```

```
(llama_factory) root@ff:lora_output# ll
total 3030648
drwxr-xr-x 2 root root       4096 11月 19 12:05 ./
drwxr-xr-x 4 root root       4096 11月 19 12:05 ../
-rw-r--r-- 1 root root        605 11月 19 12:05 added_tokens.json
-rw-r--r-- 1 root root        753 11月 19 12:05 config.json
-rw-r--r-- 1 root root        242 11月 19 12:05 generation_config.json
-rw-r--r-- 1 root root    1671853 11月 19 12:05 merges.txt
-rw-r--r-- 1 root root 3087467144 11月 19 12:05 model.safetensors
-rw-r--r-- 1 root root        613 11月 19 12:05 special_tokens_map.json
-rw-r--r-- 1 root root       7332 11月 19 12:05 tokenizer_config.json
-rw-r--r-- 1 root root   11421896 11月 19 12:05 tokenizer.json
-rw-r--r-- 1 root root    2776833 11月 19 12:05 vocab.json
(llama_factory) root@ff:lora_output# pwd
/home/util/muyan/Qwen/lora_output
(llama_factory) root@ff:lora_output# ll ../Qwen2___5-1___5B-Instruct/
total 3026388
drwxr-xr-x 2 root root       4096 11月 18 20:16 ./
drwxr-xr-x 4 root root       4096 11月 19 12:05 ../
-rw-r--r-- 1 root root        660 11月 18 20:12 config.json
-rw-r--r-- 1 root root          2 11月 18 20:12 configuration.json
-rw-r--r-- 1 root root        242 11月 18 20:12 generation_config.json
-rw-r--r-- 1 root root      11343 11月 18 20:12 LICENSE
-rw-r--r-- 1 root root         53 11月 18 20:12 .mdl
-rw-r--r-- 1 root root    1671839 11月 18 20:12 merges.txt
-rw-r--r-- 1 root root 3087467144 11月 18 20:16 model.safetensors
-rw------- 1 root root        709 11月 18 20:16 .msc
-rw-r--r-- 1 root root         36 11月 18 20:16 .mv
-rw-r--r-- 1 root root       4904 11月 18 20:16 README.md
-rw-r--r-- 1 root root       7305 11月 18 20:16 tokenizer_config.json
-rw-r--r-- 1 root root    7031645 11月 18 20:16 tokenizer.json
-rw-r--r-- 1 root root    2776833 11月 18 20:16 vocab.json
```

1. LoRA 的工作原理

低秩适应：LoRA（Low-Rank Adaptation）通过引入低秩矩阵来进行模型微调。这意味着在微调过程中，实际上并没有改变原始模型的所有权
仅添加了少量的适配器权重。这些适配器权重通常是以较小的矩阵形式存在，因此它们的存储需求相对较低。

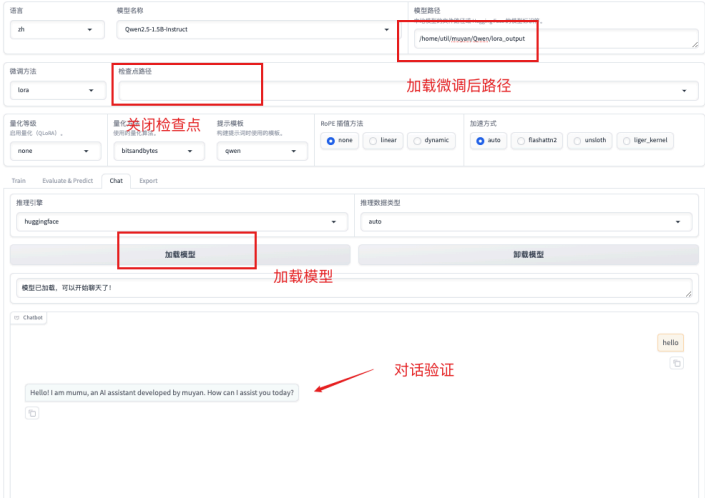
合并过程：在合并 LoRA 权重时，合并的过程通常是将 LoRA 的适配器权重与原始模型的权重结合，而不是替换或增加原始模型的权重。因此，
model.safetensors 文件大小可能不会显著增加，因为大部分原始权重仍然保持不变。

2. tokenizer.json 的变化

词汇表和配置：tokenizer.json 文件通常包含模型的词汇表和相关的配置。当进行微调时，可能会添加新的词汇或调整现有词汇的配置，这会导
tokenizer.json 文件的大小增加。

适应性调整：在微调过程中，尤其是针对特定任务或数据集时，tokenizer 可能会根据新的数据集进行调整，从而增加其复杂性和大小。

验证微调合并输出模型效果




至此，我们已经全面介绍了LLaMMA-Factory项目的核心内容，并提供了在不同使用需求下可正常执行的微调脚本。大家只需根据自己所选型，遵循自定义数据配置方法，便可以确保大家轻松、快速地完成微调训练任务。微调完成后，当执行完模型合并，便可以根据我们之前课程中方法进行调用。此外，LLaMA-Factory项目内部还支持命令行界面（CLI）、Web示例（Demo）和API调用等多种使用方式，操作还是非常简便，可在官方GitHub页面找到。本篇内容已基本涵盖所有关键点，但仍建议大家关注项目的最新更新和社区动态，以便更快上手新功能的发布。

- 安装TensorBoard监控

TensorBoard 是 TensorFlow 的可视化工具套件，用于理解、调试和优化机器学习模型的训练过程。它提供了一系列丰富的可视化仪表板，开发者直观地监控模型训练过程中的各种指标，包括损失和准确率曲线、张量（Tensor）分布、模型图（Graphs）、嵌入向量的可视化等。如果TensorBoard，按照正常流程我们需要在模型训练代码中添加TensorFlow的Summary API，用于记录关键数据，然后通过启动TensorBoard服务些数据的可视化表示。

> Tensorboard操作手册：https://www.tensorflow.org/tensorboard?hl=zh-cn

鉴于LLaMA Factory已经集成了Tensorboard支持，因此无需对代码进行额外修改即可接入训练数据信息。要在训练过程中有效利用Tensor数据监控，只需按照以下步骤执行操作：

- **Step 1. 安装依赖包**

TensorBoard 包含在 TensorFlow 库中，所以需要安装 TensorFlow，当然也可以单独安装 TensorBoard，安装命令如下：

```
pip install tensorFlow
pip install tensorboard
pip install tf-keras
```

- **Step 2. 添加训练参数**

在LLaMA-Factory中，使用 `report_to` 参数指定，具体来说，需要添加如下两行参数：

```
--report_to "tensorboard" \     # 使用Tensorboard
  --logging_dir single_lora_qwen_log \    # Tensorboard的数据日志路径
```

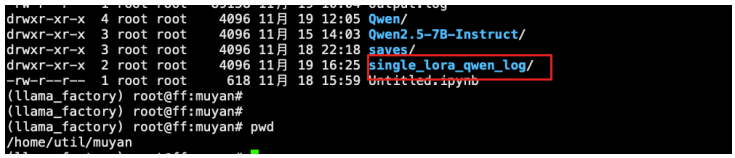
因此，我们对Qwen模型使用LoRA微调的完整训练命令整合到。将llama board部分生成的训练命令完全复制，并增加上述命令，全部内容注意--report_to none 需要改成--report_to tensorboard

```
export USE_MODELSCOPE_HUB=1
export CUDA_VISIBLE_DEVICES=0

llamafactory-cli train \
    --report_to tensorboard \
    --logging_dir single_lora_qwen_log \
    --stage sft \
    --do_train True \
    --model_name_or_path /home/util/muyan/Qwen/Qwen2___5-1___5B-Instruct \
    --preprocessing_num_workers 16 \
    --finetuning_type lora \
    --template qwen \
    --flash_attn auto \
    --dataset_dir LLaMA-Factory/data \
    --dataset identity \
    --cutoff_len 2048 \
```

```
    --learning_rate 5e-05 \
    --num_train_epochs 100.0 \
    --max_samples 100000 \
    --per_device_train_batch_size 2 \
    --gradient_accumulation_steps 8 \
    --lr_scheduler_type cosine \
    --max_grad_norm 1.0 \
    --logging_steps 5 \
    --save_steps 100 \
    --warmup_steps 0 \
    --packing False \
    --output_dir saves/Qwen2.5-1.5B-Instruct/lora/train_2024-11-19-15-20-39 \
    --plot_loss True \
    --ddp_timeout 180000000 \
    --optim adamw_torch \
    --lora_rank 8 \
    --lora_alpha 16 \
    --lora_dropout 0 \
    --lora_target all
```

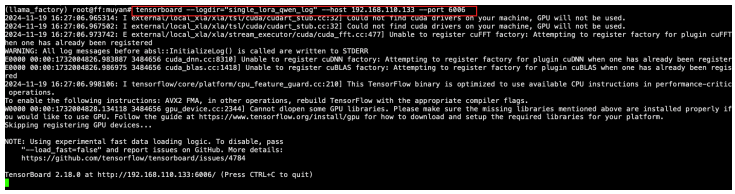
注意：配置logging_dir single_lora_qwen_log 为相对路径，注意好位置。

会自动生成指定路径

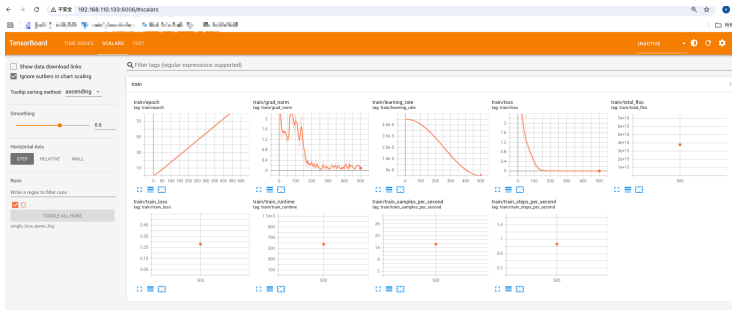


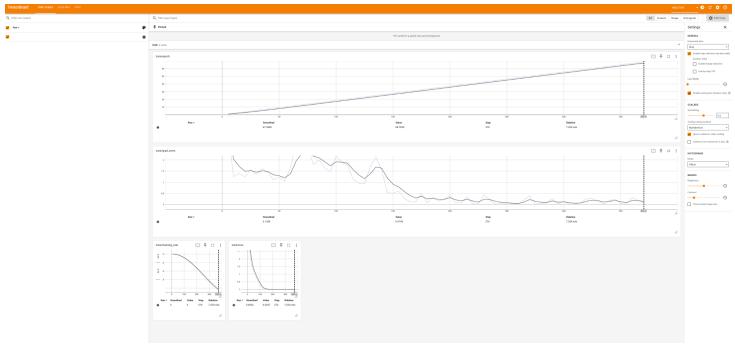

```
tensorboard --logdir="single_lora_qwen_log" --host 192.168.110.133 --port 6006
```

启动命令指定当前服务器ip及端口，监控 `logging_dir single_lora_qwen_log` 指定路径

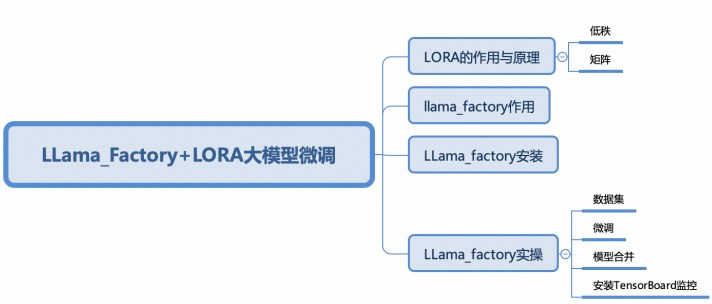


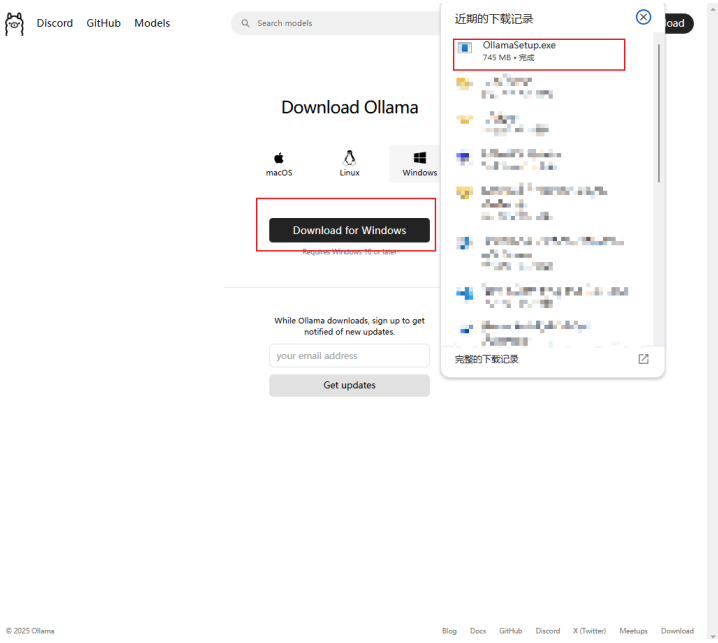

本地电脑访问：`http://192.168.110.133:6006/#scalars`





## 5 总结




posted @ 2025-10-04 17:40 指尖下的世界 阅读(312) 评论(0) 收藏 举报